

ExtraCredit1

December 11, 2025

1 DSCC 201/401 Final Project - Question1

EXTRA CREDIT 1 (10 points): Use the Keras library as provided by the Python 3 (miniforge 23.3.1-1 dsc-2024) kernel to build a simple artificial neural network for the prediction of the solar radio flux based on the sunspot number and the individual number of X-ray and optical solar flares. Begin with importing your cleaned data file (solar_data_clean.csv) into a Panadas data frame. Partition the data set appropriately and train your neural network. What is the predicted radio flux for a day when 73 sunspots are observed on the sun and 2 solar flares with C-class energy levels and 1 solar flare with an M-class energy level occur? How confident are you in the answer Explain your reasoning. Please provide a PDF of your Jupyter notebook showing ALL input and output and upload BOTH the PDF and the Jupyter notebook file (ExtraCredit1.ipynb and ExtraCredit1.pdf) showing all input and output.

```
[5]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Load the cleaned data file
df = pd.read_csv("solar_data_clean.csv")
df.head()
```

```
[5]:
```

	Date	Radio_Flux_10.7_cm	SESC_Sunspot_Number	Sunspot_Area	\
0	1997-01-01	72	0	0	
1	1997-01-02	72	0	0	
2	1997-01-03	73	0	0	
3	1997-01-04	74	13	10	
4	1997-01-05	74	15	20	

	New_Regions	GOES15_XRay_Bkgd_Flux	XRay_C	XRay_M	XRay_X	S	Optical_1	\
0	0	A0.5	0	0	0	1	0	
1	0	A0.5	0	0	0	1	0	
2	0	A0.5	0	0	0	1	0	

3	1	A0.6	0	0	0	1	0
4	0	A0.7	0	0	0	1	0

	Optical_2	Optical_3
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

```
[6]: # Select feature and target columns
feature_cols = [
    "SESC_Sunspot_Number",
    "XRay_C",
    "XRay_M",
    "XRay_X",
    "S",
    "Optical_1",
    "Optical_2",
    "Optical_3"
]

target_col = "Radio_Flux_10.7_cm"

X = df[feature_cols].to_numpy()
y = df[target_col].to_numpy()

# Partition the data set into training set and test set
# standardize the features
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Build a neural network model
n_features = X_train_scaled.shape[1]

model = keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1)
])
```

```

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"]
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	288
dense_1 (Dense)	(None, 16)	528
dense_2 (Dense)	(None, 1)	17

Total params: 833 (3.25 KB)

Trainable params: 833 (3.25 KB)

Non-trainable params: 0 (0.00 B)

```

[7]: # Train the neural network model
model.fit(
    X_train_scaled, y_train,
    validation_data=(X_test_scaled, y_test),
    epochs=100,
    batch_size=32,
    verbose=1
)

```

```

Epoch 1/100
264/264          1s 1ms/step -
loss: 13593.4980 - mae: 107.9848 - val_loss: 3775.2136 - val_mae: 53.0760
Epoch 2/100
264/264          0s 1ms/step -
loss: 2205.4619 - mae: 36.0957 - val_loss: 722.1401 - val_mae: 18.8043
Epoch 3/100
264/264          0s 1ms/step -
loss: 653.0989 - mae: 17.8400 - val_loss: 517.8265 - val_mae: 15.4959
Epoch 4/100

```

264/264 0s 1ms/step -
loss: 512.2719 - mae: 15.1984 - val_loss: 416.5073 - val_mae: 13.6283
Epoch 5/100

264/264 0s 1ms/step -
loss: 431.1604 - mae: 13.5991 - val_loss: 354.3467 - val_mae: 12.7184
Epoch 6/100

264/264 0s 1ms/step -
loss: 372.7403 - mae: 12.6185 - val_loss: 310.7935 - val_mae: 11.7431
Epoch 7/100

264/264 0s 1ms/step -
loss: 321.0659 - mae: 11.8151 - val_loss: 288.4671 - val_mae: 11.3131
Epoch 8/100

264/264 0s 1ms/step -
loss: 311.4010 - mae: 11.6424 - val_loss: 273.4472 - val_mae: 11.0063
Epoch 9/100

264/264 0s 1ms/step -
loss: 299.2108 - mae: 11.3826 - val_loss: 262.6357 - val_mae: 10.8366
Epoch 10/100

264/264 0s 1ms/step -
loss: 295.3367 - mae: 11.2841 - val_loss: 257.7208 - val_mae: 10.8239
Epoch 11/100

264/264 0s 1ms/step -
loss: 290.5471 - mae: 11.2593 - val_loss: 250.4081 - val_mae: 10.7099
Epoch 12/100

264/264 0s 1ms/step -
loss: 294.3449 - mae: 11.1446 - val_loss: 246.4851 - val_mae: 10.6396
Epoch 13/100

264/264 0s 1ms/step -
loss: 307.5171 - mae: 11.2592 - val_loss: 242.6508 - val_mae: 10.5155
Epoch 14/100

264/264 0s 1ms/step -
loss: 262.9943 - mae: 10.8161 - val_loss: 241.2265 - val_mae: 10.5417
Epoch 15/100

264/264 0s 1ms/step -
loss: 289.6469 - mae: 11.0958 - val_loss: 236.8069 - val_mae: 10.4460
Epoch 16/100

264/264 0s 1ms/step -
loss: 277.1552 - mae: 11.2707 - val_loss: 236.3907 - val_mae: 10.3869
Epoch 17/100

264/264 0s 1ms/step -
loss: 281.6618 - mae: 11.1903 - val_loss: 233.0253 - val_mae: 10.3366
Epoch 18/100

264/264 0s 1ms/step -
loss: 257.0105 - mae: 10.8832 - val_loss: 234.7144 - val_mae: 10.5165
Epoch 19/100

264/264 0s 1ms/step -
loss: 251.4990 - mae: 10.7635 - val_loss: 232.7700 - val_mae: 10.2603
Epoch 20/100

264/264 0s 1ms/step -
 loss: 269.4815 - mae: 11.0622 - val_loss: 230.9422 - val_mae: 10.3090
 Epoch 21/100
 264/264 0s 1ms/step -
 loss: 277.7361 - mae: 11.1082 - val_loss: 229.7995 - val_mae: 10.3291
 Epoch 22/100
 264/264 0s 1ms/step -
 loss: 256.4807 - mae: 10.7624 - val_loss: 228.3055 - val_mae: 10.2694
 Epoch 23/100
 264/264 0s 1ms/step -
 loss: 257.9724 - mae: 10.7582 - val_loss: 227.0386 - val_mae: 10.1735
 Epoch 24/100
 264/264 0s 1ms/step -
 loss: 247.2789 - mae: 10.5505 - val_loss: 228.7395 - val_mae: 10.2483
 Epoch 25/100
 264/264 0s 1ms/step -
 loss: 261.9708 - mae: 10.7291 - val_loss: 225.7786 - val_mae: 10.3136
 Epoch 26/100
 264/264 0s 1ms/step -
 loss: 243.9772 - mae: 10.5443 - val_loss: 224.8715 - val_mae: 10.2685
 Epoch 27/100
 264/264 0s 1ms/step -
 loss: 248.6325 - mae: 10.5589 - val_loss: 224.7048 - val_mae: 10.1815
 Epoch 28/100
 264/264 0s 1ms/step -
 loss: 254.7072 - mae: 10.7401 - val_loss: 222.2884 - val_mae: 10.1927
 Epoch 29/100
 264/264 0s 1ms/step -
 loss: 241.3980 - mae: 10.5807 - val_loss: 222.4919 - val_mae: 10.0590
 Epoch 30/100
 264/264 0s 1ms/step -
 loss: 248.3858 - mae: 10.6332 - val_loss: 220.8158 - val_mae: 10.1723
 Epoch 31/100
 264/264 0s 1ms/step -
 loss: 257.9799 - mae: 10.7043 - val_loss: 220.1872 - val_mae: 10.1680
 Epoch 32/100
 264/264 0s 1ms/step -
 loss: 255.7690 - mae: 10.7930 - val_loss: 220.2283 - val_mae: 10.0885
 Epoch 33/100
 264/264 0s 1ms/step -
 loss: 267.4357 - mae: 10.6761 - val_loss: 220.4318 - val_mae: 10.1555
 Epoch 34/100
 264/264 0s 1ms/step -
 loss: 239.5623 - mae: 10.4892 - val_loss: 218.2655 - val_mae: 10.0342
 Epoch 35/100
 264/264 0s 1ms/step -
 loss: 253.9495 - mae: 10.6025 - val_loss: 220.9506 - val_mae: 10.0681
 Epoch 36/100

264/264 0s 1ms/step -
 loss: 245.4612 - mae: 10.5585 - val_loss: 218.9483 - val_mae: 10.0571
 Epoch 37/100
 264/264 0s 1ms/step -
 loss: 244.6655 - mae: 10.5026 - val_loss: 221.2869 - val_mae: 10.1693
 Epoch 38/100
 264/264 0s 1ms/step -
 loss: 239.3253 - mae: 10.5251 - val_loss: 223.1641 - val_mae: 10.2967
 Epoch 39/100
 264/264 0s 1ms/step -
 loss: 245.9436 - mae: 10.5899 - val_loss: 219.6405 - val_mae: 10.0666
 Epoch 40/100
 264/264 0s 1ms/step -
 loss: 248.6773 - mae: 10.5015 - val_loss: 217.6795 - val_mae: 10.0540
 Epoch 41/100
 264/264 0s 1ms/step -
 loss: 246.5476 - mae: 10.4890 - val_loss: 217.5472 - val_mae: 10.0665
 Epoch 42/100
 264/264 0s 1ms/step -
 loss: 236.9153 - mae: 10.3533 - val_loss: 218.0190 - val_mae: 10.0483
 Epoch 43/100
 264/264 0s 1ms/step -
 loss: 239.0289 - mae: 10.3831 - val_loss: 220.7057 - val_mae: 10.3056
 Epoch 44/100
 264/264 0s 1ms/step -
 loss: 242.8641 - mae: 10.4798 - val_loss: 225.4191 - val_mae: 10.3092
 Epoch 45/100
 264/264 0s 1ms/step -
 loss: 239.3108 - mae: 10.4439 - val_loss: 224.4539 - val_mae: 10.2816
 Epoch 46/100
 264/264 0s 1ms/step -
 loss: 247.9648 - mae: 10.5715 - val_loss: 223.7772 - val_mae: 10.4056
 Epoch 47/100
 264/264 0s 1ms/step -
 loss: 238.8676 - mae: 10.5176 - val_loss: 216.4035 - val_mae: 10.0307
 Epoch 48/100
 264/264 0s 1ms/step -
 loss: 239.2308 - mae: 10.3226 - val_loss: 218.3362 - val_mae: 10.0485
 Epoch 49/100
 264/264 0s 1ms/step -
 loss: 234.8552 - mae: 10.4013 - val_loss: 219.9002 - val_mae: 10.2116
 Epoch 50/100
 264/264 0s 1ms/step -
 loss: 244.7202 - mae: 10.5041 - val_loss: 218.2897 - val_mae: 10.0894
 Epoch 51/100
 264/264 0s 1ms/step -
 loss: 234.0221 - mae: 10.3866 - val_loss: 218.2570 - val_mae: 10.0505
 Epoch 52/100

264/264 0s 1ms/step -
 loss: 231.2264 - mae: 10.2524 - val_loss: 221.4946 - val_mae: 10.3353
 Epoch 53/100
 264/264 0s 1ms/step -
 loss: 236.6323 - mae: 10.4207 - val_loss: 217.2928 - val_mae: 10.0700
 Epoch 54/100
 264/264 0s 1ms/step -
 loss: 250.8074 - mae: 10.3253 - val_loss: 218.9718 - val_mae: 10.0902
 Epoch 55/100
 264/264 0s 1ms/step -
 loss: 246.0785 - mae: 10.5286 - val_loss: 216.1222 - val_mae: 10.0588
 Epoch 56/100
 264/264 0s 1ms/step -
 loss: 248.8585 - mae: 10.5884 - val_loss: 220.9793 - val_mae: 10.2781
 Epoch 57/100
 264/264 0s 1ms/step -
 loss: 245.3412 - mae: 10.4813 - val_loss: 216.1863 - val_mae: 10.0186
 Epoch 58/100
 264/264 0s 1ms/step -
 loss: 235.7119 - mae: 10.1759 - val_loss: 218.7772 - val_mae: 10.1314
 Epoch 59/100
 264/264 0s 1ms/step -
 loss: 231.9564 - mae: 10.3488 - val_loss: 219.8473 - val_mae: 10.1183
 Epoch 60/100
 264/264 0s 1ms/step -
 loss: 242.5734 - mae: 10.4974 - val_loss: 217.5179 - val_mae: 9.9984
 Epoch 61/100
 264/264 0s 1ms/step -
 loss: 242.4652 - mae: 10.5186 - val_loss: 219.8336 - val_mae: 10.2551
 Epoch 62/100
 264/264 0s 1ms/step -
 loss: 252.6155 - mae: 10.7240 - val_loss: 217.7517 - val_mae: 10.0630
 Epoch 63/100
 264/264 0s 1ms/step -
 loss: 235.5392 - mae: 10.3752 - val_loss: 218.0486 - val_mae: 10.0556
 Epoch 64/100
 264/264 0s 1ms/step -
 loss: 234.0461 - mae: 10.3489 - val_loss: 219.6178 - val_mae: 10.2165
 Epoch 65/100
 264/264 0s 1ms/step -
 loss: 243.8460 - mae: 10.5385 - val_loss: 220.1291 - val_mae: 10.1542
 Epoch 66/100
 264/264 0s 1ms/step -
 loss: 233.9319 - mae: 10.3754 - val_loss: 215.9424 - val_mae: 10.0500
 Epoch 67/100
 264/264 0s 1ms/step -
 loss: 238.1605 - mae: 10.5406 - val_loss: 220.6050 - val_mae: 10.1999
 Epoch 68/100

264/264 0s 1ms/step -
 loss: 268.4592 - mae: 10.8000 - val_loss: 217.8681 - val_mae: 10.1991
 Epoch 69/100
 264/264 0s 1ms/step -
 loss: 249.5453 - mae: 10.7148 - val_loss: 215.8949 - val_mae: 10.0313
 Epoch 70/100
 264/264 0s 1ms/step -
 loss: 243.4126 - mae: 10.5505 - val_loss: 217.4775 - val_mae: 10.0537
 Epoch 71/100
 264/264 0s 1ms/step -
 loss: 239.3642 - mae: 10.4838 - val_loss: 216.1699 - val_mae: 10.0058
 Epoch 72/100
 264/264 0s 1ms/step -
 loss: 243.1548 - mae: 10.5524 - val_loss: 216.6281 - val_mae: 10.0238
 Epoch 73/100
 264/264 0s 1ms/step -
 loss: 236.3328 - mae: 10.4714 - val_loss: 216.9380 - val_mae: 10.0905
 Epoch 74/100
 264/264 0s 1ms/step -
 loss: 247.1243 - mae: 10.4858 - val_loss: 218.2319 - val_mae: 10.1167
 Epoch 75/100
 264/264 0s 1ms/step -
 loss: 236.4379 - mae: 10.4144 - val_loss: 218.0829 - val_mae: 10.2138
 Epoch 76/100
 264/264 0s 1ms/step -
 loss: 241.1236 - mae: 10.4422 - val_loss: 216.0891 - val_mae: 10.0281
 Epoch 77/100
 264/264 0s 1ms/step -
 loss: 234.4794 - mae: 10.3266 - val_loss: 216.9164 - val_mae: 10.1178
 Epoch 78/100
 264/264 0s 1ms/step -
 loss: 235.0234 - mae: 10.4008 - val_loss: 216.1388 - val_mae: 10.0093
 Epoch 79/100
 264/264 0s 1ms/step -
 loss: 256.2812 - mae: 10.5356 - val_loss: 215.2703 - val_mae: 10.0077
 Epoch 80/100
 264/264 0s 1ms/step -
 loss: 237.5309 - mae: 10.3878 - val_loss: 218.8731 - val_mae: 10.1049
 Epoch 81/100
 264/264 0s 1ms/step -
 loss: 239.3270 - mae: 10.4558 - val_loss: 215.4172 - val_mae: 10.0639
 Epoch 82/100
 264/264 0s 1ms/step -
 loss: 249.9495 - mae: 10.5194 - val_loss: 218.0159 - val_mae: 9.9624
 Epoch 83/100
 264/264 0s 1ms/step -
 loss: 251.3866 - mae: 10.7232 - val_loss: 215.9981 - val_mae: 10.0384
 Epoch 84/100

264/264 0s 1ms/step -
 loss: 248.1798 - mae: 10.4362 - val_loss: 216.3035 - val_mae: 10.0770
 Epoch 85/100
 264/264 0s 1ms/step -
 loss: 235.9847 - mae: 10.4319 - val_loss: 217.6631 - val_mae: 10.0779
 Epoch 86/100
 264/264 0s 1ms/step -
 loss: 230.2570 - mae: 10.3117 - val_loss: 214.6368 - val_mae: 10.0457
 Epoch 87/100
 264/264 0s 1ms/step -
 loss: 229.8742 - mae: 10.3621 - val_loss: 217.3605 - val_mae: 10.2789
 Epoch 88/100
 264/264 0s 1ms/step -
 loss: 236.9148 - mae: 10.3803 - val_loss: 218.2355 - val_mae: 10.1793
 Epoch 89/100
 264/264 0s 1ms/step -
 loss: 246.5326 - mae: 10.5498 - val_loss: 216.9205 - val_mae: 10.0352
 Epoch 90/100
 264/264 0s 1ms/step -
 loss: 225.2009 - mae: 10.1614 - val_loss: 216.8401 - val_mae: 10.1476
 Epoch 91/100
 264/264 0s 1ms/step -
 loss: 234.0422 - mae: 10.2689 - val_loss: 215.2460 - val_mae: 9.9410
 Epoch 92/100
 264/264 0s 1ms/step -
 loss: 235.8353 - mae: 10.4284 - val_loss: 215.0495 - val_mae: 10.0144
 Epoch 93/100
 264/264 0s 1ms/step -
 loss: 230.7824 - mae: 10.3565 - val_loss: 216.4985 - val_mae: 9.9991
 Epoch 94/100
 264/264 0s 1ms/step -
 loss: 244.6297 - mae: 10.5687 - val_loss: 214.6361 - val_mae: 9.9549
 Epoch 95/100
 264/264 0s 1ms/step -
 loss: 232.3129 - mae: 10.2851 - val_loss: 219.3423 - val_mae: 10.3060
 Epoch 96/100
 264/264 0s 1ms/step -
 loss: 251.1466 - mae: 10.5906 - val_loss: 219.1467 - val_mae: 10.1340
 Epoch 97/100
 264/264 0s 1ms/step -
 loss: 234.8935 - mae: 10.3475 - val_loss: 219.3057 - val_mae: 10.1853
 Epoch 98/100
 264/264 0s 1ms/step -
 loss: 234.2176 - mae: 10.3659 - val_loss: 216.9256 - val_mae: 10.1964
 Epoch 99/100
 264/264 0s 1ms/step -
 loss: 243.0244 - mae: 10.6532 - val_loss: 214.0339 - val_mae: 9.9840
 Epoch 100/100

```
264/264          0s 1ms/step -  
loss: 245.0129 - mae: 10.5545 - val_loss: 213.6401 - val_mae: 9.9537
```

```
[7]: <keras.src.callbacks.history.History at 0x2ba1d1dbc950>
```

```
[8]: # Evaluate the Model Performance on the test set  
test_mse, test_mae = model.evaluate(X_test_scaled, y_test, verbose=0)  
print("Test MSE:", test_mse)  
print("Test MAE:", test_mae)
```

```
Test MSE: 213.64012145996094  
Test MAE: 9.953743934631348
```

```
[22]: # Use the given features to predict the solar radio flux  
x_new = np.array([[73, 2, 1, 0, 0, 0, 0, 0]])  
x_new_scaled = scaler.transform(x_new)  
predicted_radio_flux = model.predict(x_new_scaled)  
print("Predicted radio flux:", predicted_radio_flux[0,0])
```

```
1/1          0s 21ms/step  
Predicted radio flux: 114.72667
```

```
[23]: # Compute 95% heuristic prediction interval  
y_pred = predicted_radio_flux[0, 0]  
margin = 1.96 * test_mae  
lower_CI = y_pred - margin  
upper_CI = y_pred + margin  
  
print(f"95% prediction interval: [{lower_CI:.2f}, {upper_CI:.2f}]")
```

```
95% prediction interval: [95.22, 134.24]
```

Using the trained neural network model, **the predicted solar radio flux** for a day when 73 sunspots are observed on the sun and 2 solar flares with C-class energy levels and 1 solar flare with an M-class energy level occur is approximately **114.73**. The neural network achieves **a MAE of approximately 9.95 on the test set**, which provides an empirical estimate of the prediction error on unseen data. Because neural networks do not provide statistical standard errors, a statistical **prediction interval cannot be constructed** as in linear regression model. Instead, the test-set MAE was used to construct **a heuristic prediction interval**, which is roughly **95.22 to 134.24**. I am **moderately confident** in the prediction.

```
[ ]:
```